A NIST-Compliant Framework for Post-Quantum Secure Aggregation in Federated Learning

Anh-Tu Tran[1⋆], Van-Quyet Nguyen[1], and The-Dung Luong[1]

Academy of Cryptography Techniques, Hanoi, Vietnam
tutran@actvn.edu.vn
quyetnv.gvm@actvn.edu.vn
thedungluong1@gmail.com

Abstract. Federated learning allows multiple clients to collaboratively train a global model without sharing raw data. However, model updates can still leak sensitive information, making secure aggregation essential for privacy protection. Existing secure aggregation protocols often rely on classical cryptographic primitives, such as Diffie–Hellman key exchange and elliptic-curve signatures, which are vulnerable to quantum attacks. This paper proposes a NIST-compliant framework for post-quantum secure aggregation in federated learning. The framework replaces classical cryptographic components with NIST-standardized post-quantum primitives, including ML-KEM for secure key establishment and ML-DSA or SLH-DSA for authentication and message integrity. It further integrates dropout-resilient aggregation through double masking and threshold secret sharing, ensuring that the server learns only the aggregated model update while individual client updates remain private. The proposed framework provides privacy against honest-but-curious and malicious adversaries, supports client dropout recovery, and offers a practical migration path toward post-quantum federated learning systems.

Keywords: Privacy Preserving Deep Learning · Post-Quantum Cryptography · Federated Learning · Secure Aggregation.

1 Introduction

Federated Learning (FL) has emerged as a promising distributed machine learning paradigm that allows multiple clients to collaboratively train a shared global model without directly exposing their raw data [9]. Instead of uploading local datasets to a central server, each client performs local training and sends only model updates, such as gradients or parameter updates, for aggregation. This architecture is particularly attractive for privacy-sensitive environments, including mobile computing, healthcare, finance, and edge intelligence [16]. However, FL alone does not fully guarantee privacy. Model updates can still leak sensitive information about local datasets through inference attacks, reconstruction attacks, or malicious server behavior. Therefore, additional privacy-preserving

⋆ Corresponding author.

mechanisms are required to protect individual client contributions during training [4].

Secure aggregation has become a key technique for addressing this problem. Its goal is to enable the server to compute only the aggregate of clients' model updates, such as their sum or average, while preventing access to any individual update [7]. Existing secure aggregation protocols can generally be divided into encryption-based and secret-sharing-based approaches [15]. Encryption-based approaches, such as those using homomorphic encryption or functional encryption [18], allow the server to compute over encrypted updates without decrypting individual client values. These methods provide strong confidentiality guarantees, but they often introduce significant computation and communication overhead. Some schemes also require shared decryption keys or trusted key-management entities, which may create additional trust assumptions and collusion risks [14]. Secret-sharing-based approaches, on the other hand, divide secrets or masking materials into shares and reconstruct only the required aggregate or dropout-recovery information when a sufficient threshold of shares is available. These approaches are often more efficient for multiparty summation and dropout recovery, but practical implementations still depend on classical cryptographic components for key establishment, authentication, and secure share distribution [8].

The emergence of quantum computing introduces a new challenge for secure aggregation in FL. Many classical secure aggregation protocols rely on public-key primitives such as Diffie–Hellman key exchange, RSA, or elliptic-curve cryptography [3]. These primitives are vulnerable to quantum attacks because their security depends on integer factorization or discrete logarithm assumptions [6]. In Bonawitz-style protocols [3, 17, 5], for example, pairwise masking seeds are established using classical key agreement. If an adversary with quantum capabilities can recover these shared secrets, the confidentiality of masked client updates may be compromised. Although secret sharing itself can offer information-theoretic privacy below the reconstruction threshold, the overall protocol may still be insecure if its surrounding cryptographic mechanisms are not post-quantum secure.

Recent studies have started to investigate post-quantum secure aggregation. PQSF proposes a lattice-based multi-stage secret sharing scheme, Improved-Pilaram, and combines it with double masking to construct a post-quantum secure privacy-preserving FL protocol [19]. This approach reduces the need to frequently update local secret shares during training and lowers computational overhead compared with existing lattice-based solutions. Another direction explores code-based secure aggregation using key- and message-additive homomorphic encryption under the Learning Parity with Noise assumption. This line of work provides an alternative to lattice-based assumptions and supports crypto-agility in post-quantum secure aggregation [2].

Despite these advances, existing works are often designed around specific cryptographic assumptions or protocol structures. A systematic framework that aligns secure aggregation with standardized post-quantum cryptographic primitives remains underdeveloped. This gap is important because practical deploy-

ment requires not only theoretical post-quantum security, but also compliance with widely accepted standards, interoperability, parameter guidance, and migration paths from existing classical systems.

To address this gap, this paper proposes a NIST-compliant framework for post-quantum secure aggregation in federated learning. The framework replaces quantum-vulnerable public-key components with NIST-standardized post-quantum primitives [1]. Specifically, ML-KEM is used for post-quantum key establishment, while ML-DSA or SLH-DSA is used for authentication and message integrity. The framework preserves the core structure of secure aggregation, including masking, double masking, threshold secret sharing, and dropout recovery, while upgrading the cryptographic foundation to support security against quantum-capable adversaries.

The proposed framework is designed with modularity and crypto-agility in mind. It can support different aggregation backends, including masking-based aggregation, lattice-based secret-sharing mechanisms, and code-based homomorphic aggregation. This design allows the system to adapt to future changes in post-quantum cryptographic standards, deployment requirements, and performance constraints. Moreover, the framework explicitly considers practical FL challenges such as high-dimensional model updates, client dropouts, collusion resistance, authentication, and communication efficiency.

The main contributions of this paper are summarized as follows:

1. We introduce a NIST-compliant post-quantum secure aggregation framework for federated learning, in which quantum-vulnerable components such as classical key establishment and authentication are replaced with standardized post-quantum primitives.
2. We design a dropout-resilient aggregation mechanism that combines double masking with threshold secret sharing, enabling the server to recover the aggregate model update even in the presence of client dropouts while preserving the privacy of active clients.
3. We provide a comprehensive security analysis of the proposed framework under multiple adversarial settings, including honest-but-curious servers, malicious servers, colluding clients, client dropouts, and quantum-capable adversaries.

The remainder of this paper is organized as follows. Section 2 defines the system model and threat model. Section 3 presents the proposed NIST-compliant post-quantum secure aggregation framework. Section 4 provides the security analysis. Section 5 evaluates performance and discusses practical deployment considerations. Finally, Section 6 concludes the paper and outlines future research directions.

2 System Model and Threat Model

This section defines the federated learning setting, cryptographic assumptions, adversarial capabilities, and security goals considered in this paper. The proposed framework targets a server-mediated federated learning system in which

clients collaboratively train a global model while keeping individual model updates private against classical and quantum-capable adversaries.

2.1 System Model

We consider a federated learning system with a central aggregation server $\mathcal{S}$ and a set of clients

$$\mathcal{U} = \{u_1, u_2, \ldots, u_n\}.$$

Each client u_i holds a private local dataset D_i, which is never uploaded to the server. In each training round r, the server selects a subset of clients $\mathcal{U}_r \subseteq \mathcal{U}$ and broadcasts the current global model w_r. Each selected client trains locally on D_i and computes an update $x_i^{(r)}$.

Let $\mathcal{A}_r \subseteq \mathcal{U}_r$ denote the clients that successfully complete the protocol in round r. The server aims to learn only the aggregate

$$X^{(r)} = \sum_{u_i \in \mathcal{A}_r} x_i^{(r)},$$

without obtaining any individual update $x_i^{(r)}$. The aggregate is then used to update the global model, e.g.,

$$w_{r+1} = w_r + \eta X^{(r)},$$

where η is a learning rate or scaling factor.

We assume a server-mediated communication model, where clients may exchange protocol messages through the server rather than communicating directly. Since clients may drop out due to network instability, limited resources, or computation failures, the system must tolerate a bounded number of dropouts while correctly aggregating the updates of surviving clients.

2.2 Threat Model and Security Goals

We consider an adversary that may control the aggregation server, a subset of clients, or the communication network. The adversary may be honest-but-curious or malicious, and may also be quantum-capable. Its objectives include learning individual client updates, manipulating the aggregation result, impersonating clients, replaying protocol messages, or disrupting the training process.

In the honest-but-curious setting, the server follows the protocol but attempts to infer private information from observed messages, including masked updates, public keys, ciphertexts, and dropout recovery data. In the malicious setting, the server may deviate from the protocol by issuing inconsistent participant lists, falsely declaring active clients as dropped, replaying old messages, or attempting to reconstruct masking material. A subset of clients may also collude with one another or with the server by sharing local secrets, masking seeds, secret shares, or protocol transcripts.

The framework assumes that the number of colluding clients and dropped clients remains within the protocol thresholds. Under this assumption, the server should learn only the aggregate update over surviving clients, while no bounded coalition should recover an honest client's individual update beyond what can be inferred from the final aggregate and its own inputs. Dropout recovery must reveal only the masking material required to remove the contribution of dropped clients, without exposing the updates of active clients.

A central security requirement is post-quantum resilience. Classical public-key mechanisms such as Diffie–Hellman, RSA, and elliptic-curve cryptography are vulnerable to quantum attacks based on Shor's algorithm [13]. Therefore, the proposed framework replaces quantum-vulnerable key establishment and authentication components with NIST-standardized post-quantum primitives.

Accordingly, the framework aims to achieve update privacy, post-quantum security, dropout robustness, collusion resistance, authentication and integrity, and aggregate correctness. That is, if honest clients follow the protocol and the survival threshold is satisfied, the server recovers the correct aggregate over active clients while learning no individual update.

3 Proposed NIST-Compliant Post-Quantum Secure Aggregation Framework

This section presents the proposed framework, which preserves the privacy and dropout robustness of classical secure aggregation while replacing quantum-vulnerable components with NIST-standardized post-quantum primitives. The framework follows a modular masking-based design: ML-KEM is used to establish pairwise masking seeds, while ML-DSA or SLH-DSA provides authentication and message integrity [11, 10, 12]. Double masking and threshold secret sharing are integrated to support dropout recovery without exposing individual client updates.

3.1 Design Overview

Let $\mathcal{U}_r = \{u_1, u_2, \ldots, u_m\}$ be the clients selected in round r. Each client u_i holds a private update $x_i^{(r)} \in \mathbb{Z}_q^d$, where d is the update dimension and q is the aggregation modulus. The server aims to compute only

$$X^{(r)} = \sum_{u_i \in \mathcal{A}_r} x_i^{(r)} \pmod{q},$$

where $\mathcal{A}_r \subseteq \mathcal{U}_r$ denotes the active clients that complete the protocol.

The framework proceeds through seven phases: client registration, round initialization, ML-KEM-based pairwise key establishment, threshold secret sharing for dropout recovery, masked update submission, dropout recovery, and final aggregation. In each round, clients derive pairwise masks from post-quantum shared secrets, add a self-mask for additional protection, and submit only masked

updates to the server. If some clients drop out, the server uses threshold shares from surviving clients to remove only the masks associated with dropped clients, then recovers the aggregate over active clients.

3.2 Cryptographic Setting

The framework use ML-KEM for shared-secret establishment, while ML-DSA or SLH-DSA is used to authenticate public keys, participant lists, masked updates, and recovery messages [11, 10, 12]. Shared secrets are processed by a key derivation function KDF and expanded by a pseudorandom generator PRG into masks matching the dimension of model updates.

The framework also uses a threshold secret sharing scheme SS for recovery material and an authenticated encryption scheme AE to protect confidential protocol messages. Each message is bound to a round identifier rid to prevent replay across training rounds. The design is modular, allowing the masking-based construction to be extended with lattice-based or code-based post-quantum aggregation backends when required.

3.3 Client Registration

Before participating in federated learning, each client u_i generates a post-quantum signature key pair

$$(sk_i^{sig}, pk_i^{sig}) \leftarrow \mathsf{Sig.KeyGen}(),$$

where Sig is instantiated by ML-DSA or SLH-DSA [10, 12]. The public verification key pk_i^{sig} is registered with the system and is used to authenticate all protocol messages sent by client u_i.

This registration phase prevents impersonation attacks and enables other participants to verify the origin and integrity of protocol messages. In practice, the registered public keys may be stored in a public directory, certificate authority, or server-maintained registry. The registry itself must be protected against tampering, and all public keys should be bound to client identities.

3.4 Round Initialization

At the beginning of each training round r, the server selects a subset of clients $\mathcal{U}_r$ and broadcasts a round initialization message:

$$M_r = (rid, \mathcal{U}_r, q, d, t, params),$$

where rid is a unique round identifier, q is the aggregation modulus, d is the update dimension, t is the reconstruction threshold, and $params$ denotes the cryptographic parameter set.

The server signs this message using its post-quantum signature key:

$$\sigma_S = \mathsf{Sig.Sign}(sk_S^{sig}, M_r).$$

Each client verifies the server signature before continuing:

$$\mathsf{Sig.Verify}(pk_S^{sig}, M_r, \sigma_S) = 1.$$

The round identifier rid is included in all subsequent computations and signatures to prevent replay attacks across different training rounds.

3.5 Post-Quantum Pairwise Key Establishment

Classical secure aggregation protocols often use Diffie–Hellman key agreement to derive pairwise masking seeds between clients. In the proposed framework, this component is replaced by ML-KEM, a NIST-standardized post-quantum key encapsulation mechanism [11].

For each pair of clients (u_i, u_j) with $i < j$, a post-quantum shared secret is established. One possible instantiation is as follows. Client u_j generates an ML-KEM key pair:

$$(pk_j^{kem}, sk_j^{kem}) \leftarrow \mathsf{MLKEM.KeyGen}().$$

Client u_j signs its KEM public key together with the round identifier:

$$\sigma_j^{kem} = \mathsf{Sig.Sign}(sk_j^{sig}, rid \parallel pk_j^{kem}).$$

The signed KEM public key is made available to other clients through the server. Client u_i verifies the signature and encapsulates a shared secret to u_j:

$$(ct_{i,j}, ss_{i,j}) \leftarrow \mathsf{MLKEM.Encaps}(pk_j^{kem}).$$

Client u_i sends the ciphertext $ct_{i,j}$ to u_j. After receiving it, client u_j decapsulates:

$$ss_{i,j} \leftarrow \mathsf{MLKEM.Decaps}(sk_j^{kem}, ct_{i,j}).$$

Both clients then derive a pairwise masking seed:

$$s_{i,j} = \mathsf{KDF}(ss_{i,j}, rid, i, j, \text{“}pairwise\_mask''\text{)}.$$

The domain-separated key derivation ensures that the same shared secret cannot be misused across different protocol purposes.

3.6 Mask Construction

Each pairwise seed $s_{i,j}$ is expanded into a high-dimensional mask using a pseudorandom generator:

$$p_{i,j} = \mathsf{PRG}(s_{i,j}) \in \mathbb{Z}_q^d.$$

For each client u_i, pairwise masks are added or subtracted according to a fixed ordering of client identities. Specifically, client u_i computes its pairwise mask as

$$P_i = \sum_{j:i<j} p_{i,j} - \sum_{j:j<i} p_{j,i} \pmod q.$$

When all participating clients submit their masked updates, the pairwise masks cancel out:

$$\sum_i P_i = 0 \pmod{q}.$$

To protect against malicious dropout manipulation, each client additionally samples a private self-mask seed:

$$b_i \leftarrow \{0,1\}^\lambda,$$

and expands it into a self-mask:

$$B_i = \mathsf{PRG}(b_i) \in \mathbb{Z}_q^d.$$

The final mask applied by client u_i is therefore

$$M_i = P_i + B_i \pmod{q}.$$

3.7 Threshold Secret Sharing for Dropout Recovery

To support dropout recovery, each client secret-shares the necessary recovery material before submitting its masked update. The recovery material may include the self-mask seed b_i and the private KEM secret key or pairwise recovery secret required to remove masks associated with dropped clients.

For each client u_i, the recovery secret is denoted by

$$R_i = (b_i, sk_i^{rec}),$$

where sk_i^{rec} represents the secret material needed to reconstruct pairwise masks involving u_i if u_i drops out.

Client u_i generates threshold shares using a secret sharing scheme:

$$\{R_{i\to j}\}_{u_j \in \mathcal{U}_r} \leftarrow \mathsf{SS.Share}(R_i, t, \mathcal{U}_r).$$

Each share $R_{i\to j}$ is encrypted and authenticated before being sent to client u_j:

$$C_{i\to j} = \mathsf{AE.Enc}(k_{i,j}^{share}, R_{i\to j}, rid),$$

where $k_{i,j}^{share}$ is derived from the ML-KEM shared secret:

$$k_{i,j}^{share} = \mathsf{KDF}(ss_{i,j}, rid, i, j, \text{“}share\_encryption''\text{)}.$$

This ensures that recovery shares are protected during transmission and can only be accessed by the intended recipient.

3.8 Masked Update Submission

After local training, each client u_i obtains its local update $x_i^{(r)} \in \mathbb{Z}_q^d$. The client computes its masked update:

$$y_i = x_i^{(r)} + P_i + B_i \pmod{q}.$$

The client signs the masked update and related metadata:

$$\sigma_i^{upd} = \mathsf{Sig.Sign}\left(sk_i^{sig}, rid \parallel y_i \parallel \mathcal{U}_r\right).$$

The client sends (y_i, σ_i^{upd}) to the server. The server verifies the signature:

$$\mathsf{Sig.Verify}\left(pk_i^{sig}, rid \parallel y_i \parallel \mathcal{U}_r, \sigma_i^{upd}\right) = 1.$$

Only updates with valid signatures are accepted for aggregation.

3.9 Dropout Recovery

Let $\mathcal{A}_r$ denote the set of clients whose masked updates are successfully received by the server, and let

$$\mathcal{D}_r = \mathcal{U}_r \setminus \mathcal{A}_r$$

denote the set of dropped clients.

If a client $u_k \in \mathcal{D}_r$ drops out before submitting its masked update, the pairwise masks involving u_k do not cancel out in the aggregate. Therefore, the server requests recovery shares from surviving clients to reconstruct the masking material associated with u_k.

Each surviving client submits the corresponding secret share for the dropped client:

$$R_{k\to j}.$$

When the server obtains at least t valid shares, it reconstructs

$$R_k = \mathsf{SS.Recon}(\{R_{k\to j}\}, t).$$

Using the reconstructed recovery material, the server removes the uncancelled pairwise masks associated with dropped clients. However, for active clients in $\mathcal{A}_r$, the self-mask seeds b_i must be revealed only through a controlled recovery step that does not expose individual updates. The use of double masking prevents the server from learning an active client's update even if it attempts to manipulate dropout status.

3.10 Final Aggregation

The server first sums all valid masked updates from active clients:

$$Y = \sum_{u_i \in \mathcal{A}_r} y_i \pmod{q}.$$

Expanding y_i, we have

$$Y = \sum_{u_i \in \mathcal{A}_r} x_i^{(r)} + \sum_{u_i \in \mathcal{A}_r} P_i + \sum_{u_i \in \mathcal{A}_r} B_i \pmod{q}.$$

The pairwise masks among active clients cancel out. The server uses dropout recovery to remove the remaining pairwise masks involving dropped clients. It also removes the aggregate of self-masks according to the protocol's recovery rule. The final result is

$$X^{(r)} = \sum_{u_i \in \mathcal{A}_r} x_i^{(r)} \pmod{q}.$$

Thus, the server obtains only the aggregate update over active clients and does not learn any individual update.

3.11 NIST-Compliance Mapping

The proposed framework is NIST-compliant in the sense that quantum-vulnerable public-key components are replaced with NIST-standardized post-quantum primitives. Table 1 summarizes the mapping between protocol components and post-quantum primitives.

Table 1. Mapping between framework components and NIST-compliant primitives.

Protocol Component	**Primitive**	**Role**
Key establishment	ML-KEM	Pairwise shared secret generation
Authentication	ML-DSA / SLH-DSA	Message and identity authentication
Mask generation	KDF + PRG	Expansion of shared secrets into masks
Share protection	AEAD	Confidential delivery of secret shares
Dropout recovery	Threshold secret sharing	Reconstruction of recovery material
Integrity protection	Hash / signature	Transcript binding and replay prevention

It is important to note that NIST has not standardized a complete secure aggregation protocol for federated learning. Rather, the proposed framework constructs secure aggregation from NIST-standardized post-quantum primitives wherever applicable. This provides a practical migration path from classical secure aggregation to post-quantum secure federated learning.

4 Security Analysis

This section analyzes the security of the proposed NIST-compliant post-quantum secure aggregation framework. We show that the framework preserves the privacy of individual client updates, supports dropout recovery, resists bounded collusion, and remains secure against quantum-capable adversaries by replacing classical public-key components with standardized post-quantum primitives.

4.1 Security Assumptions

The security of the proposed framework relies on the following assumptions.

- **Post-quantum security of ML-KEM:** The ML-KEM instantiation is assumed to be secure against quantum-capable adversaries under the underlying module-lattice hardness assumptions [11].
- **Post-quantum unforgeability of signatures:** The digital signature scheme, instantiated by ML-DSA or SLH-DSA, is assumed to be existentially unforgeable under chosen-message attacks, even against quantum-capable adversaries [10, 12].
- **Security of symmetric primitives:** The key derivation function, pseudorandom generator, authenticated encryption scheme, and hash function are assumed to provide standard security guarantees, including pseudorandomness, confidentiality, integrity, and domain separation.
- **Threshold security of secret sharing:** The threshold secret sharing scheme ensures that any set of fewer than t shares reveals no information about the shared secret, while any set of at least t valid shares can reconstruct it correctly.
- **Bounded collusion:** The number of colluding clients is assumed to be smaller than the reconstruction threshold required to recover honest clients' private recovery material.

Under these assumptions, the proposed framework aims to guarantee that the aggregation server learns only the aggregate update and no individual client update.

4.2 Privacy Against an Honest-but-Curious Server

In the honest-but-curious model, the server follows the protocol correctly but attempts to infer individual client updates from the messages it observes. The server receives masked updates of the form

$$y_i = x_i + P_i + B_i \pmod{q},$$

where x_i is the private update of client u_i, P_i is the sum of pairwise masks, and B_i is the self-mask generated from the private seed b_i.

For any active client u_i, the value y_i is computationally indistinguishable from a random vector in $\mathbb{Z}_q^d$ from the server's perspective, assuming that the

masking seeds remain hidden and the pseudorandom generator is secure. Since the pairwise masks are derived from ML-KEM shared secrets and expanded through a secure key derivation function and pseudorandom generator, the server cannot compute P_i without access to the corresponding shared secrets.

When the server sums all valid masked updates, pairwise masks between active clients cancel out by construction. The server may use recovery information only to remove the masks associated with dropped clients. As a result, the server obtains

$$X = \sum_{u_i \in \mathcal{A}} x_i \pmod{q},$$

where $\mathcal{A}$ is the set of active clients. The server learns the aggregate update, but individual updates remain hidden.

Therefore, under the security of ML-KEM, the pseudorandom generator, and threshold secret sharing, the framework protects individual update privacy against an honest-but-curious server.

4.3 Privacy Against a Malicious Server

A malicious server may deviate from the protocol in order to recover individual client updates. For example, it may attempt to send inconsistent participant lists, replay old protocol messages, falsely declare an active client as dropped, or request recovery shares for an active client.

The proposed framework mitigates these attacks through authenticated protocol transcripts and double masking. Each round initialization message, participant list, KEM public key, masked update, and recovery message is bound to a unique round identifier *rid* and authenticated using post-quantum digital signatures. This prevents the server from modifying messages or replaying messages from previous rounds without detection.

A particularly important attack occurs when a malicious server falsely marks an active client as dropped and requests its recovery shares. If the server could reconstruct all masks of that client and later receive its masked update, the client's individual update could be exposed. This attack was one of the main motivations for double masking in practical secure aggregation protocols [3].

In the proposed framework, each client uses both pairwise masks and a private self-mask:

$$y_i = x_i + P_i + B_i \pmod{q}.$$

Even if the server reconstructs some pairwise masking material, the self-mask B_i continues to protect the active client's update. The recovery procedure distinguishes between dropped and active clients and releases only the recovery material required by the protocol. Moreover, signed surviving-client lists and transcript binding prevent the server from presenting inconsistent dropout views to different clients.

Thus, the malicious server cannot isolate an active client's update unless it obtains sufficient secret shares to reconstruct the client's private self-mask in violation of the threshold assumption.

4.4 Resistance to Colluding Clients

We next consider a coalition of malicious clients that may collude with the server. Colluding clients may reveal their own updates, pairwise seeds, secret shares, and protocol transcripts. The goal of the coalition is to learn the update of an honest client u_i.

For an honest active client, its masked update is protected by pairwise masks shared with other honest clients and by its self-mask. If the number of colluding clients is below the threshold t, the coalition cannot reconstruct the honest client's private recovery material from secret shares. In addition, pairwise masks established between two honest clients remain hidden from the coalition due to the security of ML-KEM.

The coalition may learn information implied by its own inputs and the final aggregate. This leakage is unavoidable in any aggregation protocol because the aggregate itself may reveal some information when the number of honest clients is small. Therefore, the security guarantee is that the adversary learns no additional information about honest clients' individual updates beyond what is inferable from the aggregate and the colluding clients' own updates.

This follows the standard privacy goal of secure aggregation: the adversary's view can be simulated using only the aggregate result, the corrupted clients' inputs, and public protocol parameters.

4.5 Dropout Robustness

Client dropout is a central challenge in federated learning. A client may drop out after key establishment, after share distribution, or before submitting its masked update. If dropout is not handled properly, the pairwise masks involving dropped clients may fail to cancel out, preventing the server from recovering the correct aggregate.

The proposed framework addresses this issue using threshold secret sharing. Each client distributes shares of its recovery material to other participants. If a client drops out, the server collects at least t valid shares from surviving clients to reconstruct the necessary recovery material and remove the uncancelled masks associated with the dropped client.

Let $\mathcal{D}$ denote the set of dropped clients and $\mathcal{A}$ denote the set of active clients. After summing the masked updates from active clients, the server obtains

$$Y = \sum_{u_i \in \mathcal{A}} x_i + \sum_{u_i \in \mathcal{A}} P_i + \sum_{u_i \in \mathcal{A}} B_i \pmod{q}.$$

The pairwise masks among active clients cancel out. The remaining uncancelled masks are those involving dropped clients. By reconstructing the recovery material of dropped clients, the server can remove these masks and recover the aggregate over the active clients.

Correctness is guaranteed as long as the number of surviving clients is sufficient to meet the reconstruction threshold. If fewer than t valid shares are available, the protocol aborts rather than compromising privacy.

4.6 Post-Quantum Security

The framework is designed to remain secure against quantum-capable adversaries. Classical secure aggregation protocols often rely on Diffie–Hellman or elliptic-curve key agreement to establish pairwise masking seeds. These mechanisms are vulnerable to Shor's algorithm and therefore do not provide long-term security in the post-quantum setting [13].

In the proposed framework, classical key agreement is replaced by ML-KEM. Pairwise masking seeds are derived from ML-KEM shared secrets:

$$s_{i,j} = \mathsf{KDF}(ss_{i,j}, rid, i, j, \text{“}pairwise\_mask\text{''}).$$

Assuming ML-KEM is secure against quantum-capable adversaries, the shared secret $ss_{i,j}$ remains hidden even if the adversary observes the public key and ciphertext.

Similarly, classical signatures are replaced by ML-DSA or SLH-DSA. This ensures post-quantum authentication of public keys, participant lists, masked updates, and recovery messages. As a result, quantum-capable adversaries cannot forge protocol messages or impersonate honest clients under the assumed security of the selected signature scheme.

Therefore, the framework upgrades the quantum-vulnerable public-key components of classical secure aggregation while preserving its masking and dropout-recovery structure.

4.7 Authentication, Integrity, and Replay Protection

Authentication and message integrity are required to prevent impersonation, message tampering, and replay attacks. In the proposed framework, all critical protocol messages are signed using post-quantum digital signatures. These include:

- client registration messages;
- KEM public keys and ciphertexts;
- server participant lists;
- masked model updates;
- dropout and recovery messages;
- surviving-client lists.

Each signed message includes the round identifier *rid*, client identity, and relevant protocol metadata. This prevents an adversary from replaying messages across different rounds or substituting messages from another client. Domain separation in the key derivation function further ensures that keys derived for one purpose, such as mask generation, cannot be reused for another purpose, such as share encryption.

4.8 Aggregate Correctness

We finally analyze correctness. If all honest clients follow the protocol and the number of surviving clients satisfies the threshold condition, the server should recover the correct aggregate over active clients.
Each active client submits

$$y_i = x_i + P_i + B_i \pmod{q}.$$

Summing over all active clients gives

$$Y = \sum_{u_i \in \mathcal{A}} x_i + \sum_{u_i \in \mathcal{A}} P_i + \sum_{u_i \in \mathcal{A}} B_i \pmod{q}.$$

The pairwise masks between active clients cancel out due to the fixed sign convention:

$$\sum_{u_i \in \mathcal{A}} P_i = 0$$

except for masks involving dropped clients. These remaining masks are removed through dropout recovery. The aggregate of self-masks is also removed according to the recovery procedure. Therefore, the server obtains

$$X = \sum_{u_i \in \mathcal{A}} x_i \pmod{q}.$$

Thus, the protocol is correct as long as enough valid shares are available for recovery and all accepted messages pass signature verification.

The above analysis shows that the proposed framework satisfies the main security goals defined in Section 4. Individual updates remain private against honest-but-curious and malicious servers due to pairwise masking, self-masking, and threshold recovery. Colluding clients cannot recover honest clients' private updates unless they exceed the threshold assumption. Dropout robustness is achieved through secret sharing, while post-quantum security is provided by ML-KEM, ML-DSA, or SLH-DSA. Consequently, the framework provides a standards-aligned path toward post-quantum secure aggregation in federated learning.

5 Performance Evaluation and Discussion

This section discusses the performance aspects of the proposed NIST-compliant post-quantum secure aggregation framework. The objective is to evaluate the additional cost introduced by post-quantum primitives and to analyze the trade-offs among privacy, post-quantum security, dropout robustness, and scalability.

5.1 Evaluation Metrics

We evaluate the proposed framework using the following metrics:

- **Computation cost:** The local computation time required by each client and the total computation time required by the server. This includes key generation, encapsulation, decapsulation, signature generation, signature verification, mask generation, secret sharing, and dropout recovery.
- **Communication cost:** The amount of data transmitted by each client and the server during one training round. This includes KEM public keys, KEM ciphertexts, digital signatures, encrypted secret shares, masked updates, and recovery messages.
- **Round complexity:** The number of communication rounds required to complete one secure aggregation round.
- **Dropout tolerance:** The maximum number or fraction of clients that may drop out while still allowing the server to recover the correct aggregate.
- **Scalability:** The ability of the framework to support a large number of clients and high-dimensional model updates.
- **Security level:** The post-quantum security strength provided by the selected ML-KEM and signature parameter sets.

These metrics allow us to compare the proposed framework with classical secure aggregation protocols, encryption-based approaches, and existing post-quantum secure aggregation schemes.

5.2 Computation Overhead

The main computational overhead of the proposed framework comes from post-quantum key establishment and post-quantum authentication. In each training round, clients perform ML-KEM operations to establish shared secrets and use ML-DSA or SLH-DSA to authenticate protocol messages. Compared with classical Diffie–Hellman or elliptic-curve-based mechanisms, post-quantum primitives generally require larger keys, ciphertexts, and signatures, and may introduce additional computation.

For each client u_i, the computation cost can be decomposed as

$$C_i = C_{\mathsf{KEM}} + C_{\mathsf{Sig}} + C_{\mathsf{PRG}} + C_{\mathsf{SS}} + C_{\mathsf{AE}},$$

where C_{KEM} denotes the cost of ML-KEM operations, C_{Sig} denotes the cost of signature generation and verification, C_{PRG} denotes the cost of expanding masking seeds into high-dimensional masks, C_{SS} denotes the cost of secret sharing, and C_{AE} denotes the cost of encrypting and decrypting secret shares.

In high-dimensional federated learning, the cost of mask generation may dominate the cost of cryptographic setup, especially when the model update dimension d is large. However, PRG expansion is typically efficient and can be parallelized. Post-quantum key establishment and signature operations mainly

affect the setup and authentication phases rather than the vector aggregation itself.

The server's computation cost is mainly determined by signature verification, aggregation, and dropout recovery. If m clients participate in a round, the server verifies $O(m)$ signatures and aggregates m masked update vectors of dimension d. In the presence of dropouts, the server also reconstructs recovery secrets using threshold secret sharing. Therefore, the server-side cost can be expressed as

$$C_{\mathcal{S}} = O(m \cdot C_{\mathsf{Verify}}) + O(md) + O(|\mathcal{D}| \cdot C_{\mathsf{Recon}}),$$

where $|\mathcal{D}|$ is the number of dropped clients and C_{Recon} is the cost of reconstructing one secret from threshold shares.

5.3 Communication Overhead

The communication overhead of the proposed framework is higher than that of classical secure aggregation because post-quantum public keys, ciphertexts, and signatures are generally larger than their classical counterparts. For each client, the total communication cost in one round can be approximated as

$$T_i = T_{\mathsf{pk}} + T_{\mathsf{ct}} + T_{\mathsf{sig}} + T_{\mathsf{share}} + T_{\mathsf{update}},$$

where T_{pk} is the size of transmitted KEM public keys, T_{ct} is the size of KEM ciphertexts, T_{sig} is the size of digital signatures, T_{share} is the size of encrypted secret shares, and T_{update} is the size of the masked model update.

In many FL workloads, the model update vector is much larger than cryptographic metadata. Therefore, for large models, the relative overhead of ML-KEM public keys and ciphertexts may be moderate compared with the total update size. However, for small models or low-bandwidth environments, post-quantum metadata may become a significant part of total communication.

The pairwise key establishment phase can introduce $O(m^2)$ communication if every client establishes pairwise secrets with every other participant. This is similar to classical Bonawitz-style secure aggregation. To improve scalability, several optimizations can be considered, such as subgroup-based aggregation, committee-based recovery, or hierarchical aggregation. These techniques reduce the number of pairwise interactions and can be integrated into the proposed modular framework.

5.4 Round Complexity

The proposed framework follows the multi-round structure of practical secure aggregation. A typical instantiation requires the following communication phases:

1. server broadcasts the selected participant list and round parameters;
2. clients exchange or publish post-quantum key establishment material;
3. clients distribute encrypted secret shares for dropout recovery;
4. clients submit masked model updates;

5. surviving clients provide recovery shares if dropouts occur;
6. server computes the final aggregate.

The number of rounds is comparable to classical dropout-resilient secure aggregation protocols. The use of ML-KEM and post-quantum signatures increases message sizes but does not fundamentally change the interaction pattern. If no dropout occurs, the recovery phase can be skipped or minimized. If dropouts occur, the server performs the recovery step by collecting sufficient shares from surviving clients.

5.5 Dropout Tolerance

Dropout tolerance is controlled by the threshold parameter t. If the number of surviving clients is at least t, the server can reconstruct the recovery material needed to remove masks associated with dropped clients. If fewer than t valid shares are available, the protocol aborts to preserve privacy.

Let $m = |\mathcal{U}_r|$ be the number of selected clients in round r, and let $a = |\mathcal{A}_r|$ be the number of active clients that successfully complete the protocol. Correct aggregation is possible if

$$a \geq t.$$

The choice of t determines the trade-off between privacy and robustness. A larger threshold improves resistance against collusion because more shares are required to reconstruct a secret. However, it reduces dropout tolerance because more clients must remain active. Conversely, a smaller threshold improves robustness against dropout but weakens collusion resistance.

Therefore, t should be chosen according to the expected dropout rate and adversarial model of the FL deployment. In mobile FL, where dropout rates can be high, the threshold may need to be lower. In enterprise or cross-silo FL, where participants are more stable, a higher threshold can be used to improve security.

5.6 Comparison with Existing Approaches

Compared with encryption-based secure aggregation, the proposed masking-based framework avoids performing expensive homomorphic operations over high-dimensional model updates. Instead, it relies mainly on PRG-generated masks and modular addition, which are efficient for large vectors. This makes the framework more suitable for resource-constrained clients.

Compared with classical secret-sharing-based secure aggregation, the proposed framework introduces additional overhead from ML-KEM and post-quantum signatures. However, this overhead is necessary to provide post-quantum security. Classical protocols based on Diffie–Hellman or elliptic-curve signatures cannot provide long-term protection against quantum-capable adversaries.

Compared with existing post-quantum secure aggregation protocols, the proposed framework emphasizes NIST compliance and modularity. Lattice-based schemes such as PQSF provide efficient post-quantum secret sharing and reduce

the need to frequently update local shares [19]. Code-based schemes based on homomorphic encryption offer an alternative family of post-quantum assumptions and improve crypto-agility [2]. The proposed framework can incorporate these approaches as alternative backends while using NIST-standardized primitives for key establishment and authentication.

5.7 Parameter Selection

Parameter selection should consider both security and performance. The following aspects are particularly important:

- **ML-KEM parameter set:** ML-KEM-512, ML-KEM-768, and ML-KEM-1024 provide different security levels and performance trade-offs. Higher security levels increase key and ciphertext sizes.
- **Signature scheme:** ML-DSA offers efficient post-quantum signatures based on module lattices, while SLH-DSA provides a stateless hash-based alternative. SLH-DSA may be useful when diversity of cryptographic assumptions is desired.
- **Aggregation modulus:** The modulus q should be large enough to avoid overflow when summing quantized model updates from multiple clients.
- **Threshold value:** The reconstruction threshold t should balance dropout tolerance and collusion resistance.
- **Model update dimension:** The dimension d directly affects the size of masked updates and the cost of PRG expansion and aggregation.
- **Client group size:** The number of clients per aggregation round affects pairwise key establishment cost, secret share distribution, and server verification workload.

In practice, parameter selection should be deployment-specific. Mobile FL may prioritize low communication and dropout tolerance, while cross-silo FL may prioritize higher security levels and stronger authentication.

6 Conclusion and Future Work

This paper presented a NIST-compliant framework for post-quantum secure aggregation in federated learning. The framework replaces quantum-vulnerable primitives, such as Diffie–Hellman, RSA, and elliptic-curve cryptography, with NIST-standardized mechanisms, including ML-KEM for key establishment and ML-DSA or SLH-DSA for authentication. By combining pairwise masking, double masking, and threshold secret sharing, it preserves update privacy and supports dropout recovery without revealing active clients' individual updates.

The security analysis shows that the framework protects against honest-but-curious servers, malicious servers, colluding clients, client dropouts, and quantum-capable adversaries. At the same time, the performance discussion highlights the main trade-off between stronger post-quantum security and increased communication overhead.

Future work will focus on implementation and benchmarking in realistic FL settings, including different post-quantum parameter sets, client scales, model dimensions, and dropout rates. Further optimization of key establishment, share distribution, and lattice-based or code-based backends will be explored to improve scalability and crypto-agility.

Acknowledgments. The article was completed with support from the Academy of Cryptography Techniques.

Declaration of open data, ethics, and conflict of interest. The data used in this study was collected from several public sources and the authors also fully referred to them. There are no potential conflicts of interest in this study.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Alagic, G., Alagic, G., Apon, D., Cooper, D., Dang, Q., Dang, T., Kelsey, J., Lichtinger, J., Liu, Y.K., Miller, C., et al.: Status report on the third round of the nist post-quantum cryptography standardization process (2022)
2. Bitzer, S., Egger, M., Liu, M., Wachter-Zeh, A.: Post-quantum secure aggregation via code-based homomorphic encryption. arXiv preprint arXiv:2601.13031 (2026)
3. Bonawitz, K., Ivanov, V., Kreuter, B., Marcedone, A., McMahan, H.B., Patel, S., Ramage, D., Segal, A., Seth, K.: Practical secure aggregation for privacy-preserving machine learning. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. p. 1175–1191. CCS '17, Association for Computing Machinery, New York, NY, USA (2017). https://doi.org/10.1145/3133956.3133982, https://doi.org/10.1145/3133956.3133982
4. Chen, J., Yan, H., Liu, Z., Zhang, M., Xiong, H., Yu, S.: When federated learning meets privacy-preserving computation. ACM Computing Surveys **56**(12), 1–36 (2024)
5. Guo, X., Liu, Z., Li, J., Gao, J., Hou, B., Dong, C., Baker, T.: V eri fl: Communication-efficient and fast verifiable aggregation for federated learning. IEEE Transactions on Information Forensics and Security **16**, 1736–1751 (2020)
6. Li, K., Yan, P.G., Cai, Q.Y.: Quantum computing and the security of public key cryptography. Fundamental Research **1**(1), 85–87 (2021)
7. Liu, Z., Guo, J., Yang, W., Fan, J., Lam, K.Y., Zhao, J.: Privacy-preserving aggregation in federated learning: A survey. IEEE Transactions on Big Data (2022)
8. Mansouri, M., Önen, M., Jaballah, W.B., Conti, M.: Sok: Secure aggregation based on cryptographic schemes for federated learning. Proceedings on Privacy Enhancing Technologies (2023)
9. McMahan, H.B., Moore, E., Ramage, D., Hampson, S., y Arcas, B.A.: Communication-efficient learning of deep networks from decentralized data. In: International Conference on Artificial Intelligence and Statistics (2016), https://api.semanticscholar.org/CorpusID:14955348

10. National Institute of Standards and Technology: Module-lattice-based digital signature standard. Tech. Rep. FIPS PUB 204, U.S. Department of Commerce, Gaithersburg, MD (August 2024). https://doi.org/10.6028/NIST.FIPS.204, https://csrc.nist.gov/pubs/fips/204/final
11. National Institute of Standards and Technology: Module-lattice-based key-encapsulation mechanism standard. Tech. Rep. FIPS PUB 203, U.S. Department of Commerce, Gaithersburg, MD (August 2024). https://doi.org/10.6028/NIST.FIPS.203, https://csrc.nist.gov/pubs/fips/203/final
12. National Institute of Standards and Technology: Stateless hash-based digital signature standard. Tech. Rep. FIPS PUB 205, U.S. Department of Commerce, Gaithersburg, MD (August 2024). https://doi.org/10.6028/NIST.FIPS.205, https://csrc.nist.gov/pubs/fips/205/final
13. Petrenko, A.: Applied Quantum Cryptanalysis. River Publishers (2023)
14. Phong, L.T., Aono, Y., Hayashi, T., Wang, L., Moriai, S.: Privacy-preserving deep learning via additively homomorphic encryption. Trans. Info. For. Sec. **13**(5), 1333–1345 (May 2018)
15. Tran, A.T., Luong, T.D., Huynh, V.N.: A comprehensive survey and taxonomy on privacy-preserving deep learning. Neurocomputing **576**, 127345 (2024)
16. Uddin, M.P., Xiang, Y., Hasan, M., Bai, J., Zhao, Y., Gao, L.: A systematic literature review of robust federated learning: Issues, solutions, and future research directions. ACM Computing Surveys **57**(10), 1–62 (2025)
17. Xu, G., Li, H., Liu, S., Yang, K., Lin, X.: Verifynet: Secure and verifiable federated learning. IEEE Transactions on Information Forensics and Security **15**, 911–926 (2019)
18. Xu, R., Baracaldo, N., Zhou, Y., Anwar, A., Ludwig, H.: Hybridalpha: An efficient approach for privacy-preserving federated learning. In: Proceedings of the 12th ACM workshop on artificial intelligence and security. pp. 13–23 (2019)
19. Zhang, X., Deng, H., Wu, R., Ren, J., Ren, Y.: Pqsf: post-quantum secure privacy-preserving federated learning. Scientific Reports **14**(1), 23553 (2024)